



ICS 344 - 01

RASHED JAFRY - 202157670

DVSA URL: <http://dvsa-website-962415228810-us-east-1.s3-website.us-east-1.amazonaws.com/>

AWS Region: us-east-1

02/05/2026

Lesson #9: Vulnerable Dependencies

Lesson #9: Vulnerable Dependencies

Part 1) Goal and Vulnerability Summary

This lesson is about vulnerable dependencies in the backend Lambda function of DVSA. The main affected component is the order-manager Lambda that is called from the REST API flow. The security impact is that the function used packages with known vulnerabilities, including one critical issue in node-serialize. This is dangerous because the backend trusts third-party code that is already known to be unsafe.

Part 2) Why This Works / Root Cause

The root cause is weak dependency management. The project still had node-serialize in package.json even after Lesson 1 fix, and the code still used node-jose only for base64url decoding. Because old and risky packages stayed in the Lambda package, npm audit reported known vulnerabilities in the deployed backend path.

Part 3) Environment and Setup

- Local DVSA repository, backend/functions/order-manager
- AWS services used: API Gateway, AWS Lambda, AWS Console
- Target backend component: order-manager Lambda function
- Tools used: terminal, rg, npm ls, npm audit, code editor, zip, AWS Console

Part 4) Reproduction Steps

1. Go to backend/functions/order-manager and search the code with rg to find node-jose and node-serialize.
2. Open order-manager.js and package.json to confirm the dependency usage and dependency list.
3. Run npm ls node-serialize node-jose to show both packages are installed in this Lambda folder
4. Run npm audit and record the findings. The audit showed a critical vulnerability in node-serialize and more issues related to node-jose
5. Edit order-manager.js. Replace `const jose = require('node-jose')` with `const base64url = require('base64url')`, then replace `jose.util.base64url.decode(...)` with `base64url.decode(...)`.
6. Remove the vulnerable packages with `npm uninstall node-serialize node-jose`, then install the replacement with `npm install base64url`.
7. Run `npm ls` and `npm audit` again. After that, zip the fixed Lambda package and upload it manually in AWS Console, then deploy the updated code.

Part 5) Evidence and Proof

FIGURE 1: Source code search showing usage of vulnerable dependencies.

```
(base) rashedjafry@rasheds-MacBook-Pro DVSA % rg --files | rg "package.json|order-manager|jwt|auth|token"
package.json
backend/functions/order-manager/order-manager.js
backend/functions/order-manager/package.json
client/src/reducers/auth.js
client/package.json
(base) rashedjafry@rasheds-MacBook-Pro DVSA % rg "node-serialize|node-jose" .
./backend/functions/order-manager/order-manager.js
3:const jose = require('node-jose');

./backend/functions/order-manager/package.json
6:  "node-jose": "2.2.0",
7:  "node-serialize": "0.0.4"
(base) rashedjafry@rasheds-MacBook-Pro DVSA % |
```

FIGURE 2: Backend Lambda code importing vulnerable dependencies.

```
() package.json  JS order-manager.js X
Users > rashedjafry > Documents > GitHub > ICS344 > DVSA > backend > functions > order-manager > JS order-manager.js > ...
1  const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
2  const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
3  const jose = require('node-jose');
4
5  const ALLOWED_ACTIONS = new Set([
6    'new', 'update', 'cancel', 'get', 'orders', 'account', 'profile',
7    'shipping', 'billing', 'complete', 'inbox', 'message', 'delete',
8    'upload', 'feedback', 'admin-orders'
9  ]);
10
11
12
13 exports.handler = (event, context, callback) => {
```

FIGURE 3: package.json listing vulnerable dependencies.

```
() package.json X  JS order-manager.js
Users > rashedjafry > Documents > GitHub > ICS344 > DVSA > backend > functions > order-manager > () package.json > ...
1  {
2    "name": "order-manager",
3    "version": "3.0.0",
4    "main": "order-manager.js",
5    "dependencies": {
6      "node-jose": "2.2.0",
7      "node-serialize": "0.0.4"
8    },
9    "scripts": {
10     "test": "echo \"Error: no test specified\" && exit 1"
11   },
12   "author": "",
13   "license": "ISC",
14   "description": ""
15 }
16
```

FIGURE 4: Installed dependency tree showing node-serialize and node-jose.

```
(base) rashedjafry@rasheds-MacBook-Pro order-manager % npm ls node-serialize node-jose
order-manager@3.0.0 /Users/rashedjafry/Documents/GitHub/ICS344/DVSA/backend/functions/order-manager
├── node-jose@2.2.0
└── node-serialize@0.0.4
```

FIGURE 5: npm audit output showing the vulnerability findings.

```
# npm audit report

node-serialize *
Severity: critical
Code Execution through IIFE in node-serialize - https://github.com/advisories/GHSA-q4v7-4rhw-9hqm
No fix available
node_modules/node-serialize

uuid <14.0.0
Severity: moderate
uuid: Missing buffer bounds check in v3/v5/v6 when buf is provided - https://github.com/advisories/GHSA-wShq-g745-h8pq
No fix available
node_modules/uuid
  node-jose *
  Depends on vulnerable versions of uuid
  node_modules/node-jose

3 vulnerabilities (2 moderate, 1 critical)

Some issues need review, and may require choosing
a different dependency.
```

The main proof is the npm audit result from the real order-manager folder. It showed node-serialize with a critical issue and node-jose depending on a vulnerable uuid version. This proves the vulnerable packages were part of the backend Lambda package and not only dead files.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the order-manager Lambda package. The correct security improvement is to remove the unused risky dependency node-serialize and stop using node-jose for simple decoding. I replaced the decoding logic with the smaller base64url package and removed both vulnerable packages from the Lambda package. This mitigation fixes the root cause because the backend no longer depends on the known vulnerable libraries.

Part 7) Code / Config Changes

File / resource changed: backend/functions/order-manager/order-manager.js

Also changed: backend/functions/order-manager/package.json and package-lock.json

Before:

```
const jose = require('node-jose');
```

```
var auth_data = jose.util.base64url.decode(token_sections[1]);
```

package.json contained "node-jose": "2.2.0" and "node-serialize": "0.0.4".

After:

```
const base64url = require('base64url');
```

```
var auth_data = base64url.decode(token_sections[1]);
```

package.json removed node-jose and node-serialize, and added base64url.

FIGURE 6: order-manager.js showing vulnerable dependency usage.

The screenshot shows a code editor with the file `order-manager.js` open. The code is written in JavaScript and uses the `require` function to load dependencies. The following lines are highlighted with red boxes:

```

1  const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
2  const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
3  const jose = require('node-jose');
4
5  const ALLOWED_ACTIONS = new Set([
6    'new', 'update', 'cancel', 'get', 'orders', 'account', 'profile',
7    'shipping', 'billing', 'complete', 'inbox', 'message', 'delete',
8    'upload', 'feedback', 'admin-orders'
9  ]);
10
11
12
13 exports.handler = (event, context, callback) => {
14   // console.log(JSON.stringify(event));
15   var req = JSON.parse(event.body);
16   var headers = JSON.parse(typeof event.headers === 'string' ? event.headers : JSON.stringify(event.headers));
17   var auth_header = headers.Authorization || headers.authorization;
18   var token_sections = auth_header.split('.');
19   var auth_data = jose.util.base64url.decode(token_sections[1]);
20   var token = JSON.parse(auth_data);
21   var user = token.username;
22   var isAdmin = false;
23

```

FIGURE 7: order-manager.js showing the dependency fix.

The screenshot shows the same code editor with the file `order-manager.js` open, but with the dependencies fixed. The following lines are highlighted with red boxes:

```

1  const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
2  const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
3  // const jose = require('node-jose');
4  const base64url = require('base64url');
5
6
7  const ALLOWED_ACTIONS = new Set([
8    'new', 'update', 'cancel', 'get', 'orders', 'account', 'profile',
9    'shipping', 'billing', 'complete', 'inbox', 'message', 'delete',
10   'upload', 'feedback', 'admin-orders'
11  ]);
12
13 function decodeBase64Url(input) {
14   input = input.replace(/-/g, '+').replace(/_/g, '/');
15   while (input.length % 4) {
16     input += '=';
17   }
18   return Buffer.from(input, 'base64').toString('utf8');
19 }
20
21 exports.handler = (event, context, callback) => {
22   // console.log(JSON.stringify(event));
23   var req = JSON.parse(event.body);
24   var headers = JSON.parse(typeof event.headers === 'string' ? event.headers : JSON.stringify(event.headers));
25   var auth_header = headers.Authorization || headers.authorization;
26   var token_sections = auth_header.split('.');
27   // var auth_data = jose.util.base64url.decode(token_sections[1]);
28   var auth_data = base64url.decode(token_sections[1]);
29   var token = JSON.parse(auth_data);

```

FIGURE 8: package.json after removing vulnerable dependencies.

```

() package.json ×
() package.json > ...
1  {
2    "name": "order-manager",
3    "version": "3.0.0",
4    "main": "order-manager.js",
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1"
7    },
8    "author": "",
9    "license": "ISC",
10   "description": "",
11   "dependencies": {
12     "base64url": "^3.0.1"
13   }
14 }
15

```

Part 8) Verification After Fix

After the change, npm is no longer showing node-serialize or node-jose in the order-manager package. The previous audit findings related to these packages were removed, and the updated zip file was deployed to the Lambda function in AWS Console. The normal DVSA flow still worked after deployment.

FIGURE 10: Dependency tree after removing vulnerable packages.

```

(base) rashedjafry@MacBookPro order-manager % npm uninstall node-serialize node-jose

removed 13 packages, and audited 1 package in 565ms

found 0 vulnerabilities
(base) rashedjafry@MacBookPro order-manager % npm install base64url

added 1 package, and audited 2 packages in 1s

found 0 vulnerabilities
(base) rashedjafry@MacBookPro order-manager % npm ls node-serialize node-jose base64url
npm audit

order-manager@3.0.0 /Users/rashedjafry/Documents/GitHub/ICS344/DVSA/backend/functions/order-manager
└─ base64url@3.0.1

found 0 vulnerabilities
(base) rashedjafry@MacBookPro order-manager %

```

FIGURE 11: Zipped remediated Lambda package for manual deployment.

```
(base) rashedjafry@MacBookPro order-manager % zip -r order-manager-fixed.zip .

adding: node_modules/ (stored 0%)
adding: node_modules/.bin/ (stored 0%)
adding: node_modules/base64url/ (stored 0%)
adding: node_modules/base64url/LICENSE (deflated 40%)
adding: node_modules/base64url/dist/ (stored 0%)
adding: node_modules/base64url/dist/base64url.js (deflated 66%)
adding: node_modules/base64url/dist/base64url.d.ts (deflated 60%)
adding: node_modules/base64url/dist/.gitkeep (stored 0%)
adding: node_modules/base64url/dist/pad-string.js (deflated 54%)
adding: node_modules/base64url/dist/pad-string.d.ts (deflated 14%)
adding: node_modules/base64url/index.js (deflated 29%)
adding: node_modules/base64url/readme.md (deflated 52%)
adding: node_modules/base64url/package.json (deflated 49%)
adding: node_modules/.package-lock.json (deflated 36%)
adding: package-lock.json (deflated 46%)
adding: package.json (deflated 37%)
adding: order-manager.js (deflated 78%)
```

FIGURE 12: Target Lambda function in AWS Console.

The screenshot displays the AWS Lambda console interface for the function named **DVSA-ORDER-MANAGER**. The top navigation bar shows the AWS logo, a search bar, and the current region (United States (N. Virginia)). The breadcrumb trail indicates the path: **Lambda > Functions > DVSA-ORDER-MANAGER**. The function's name is prominently displayed at the top, along with buttons for **Throttle**, **Copy ARN**, and **Actions**. A notification bar states: "This function belongs to an application. [Click here](#) to manage it." Below this, the **Function overview** section is active, showing a diagram of the function's architecture. The diagram includes a box for **DVSA-ORDER-MANAGER** with a **Layers** section showing **(0)** layers. An **API Gateway** trigger is connected to the function, with an **+ Add trigger** button. To the right, the **Related functions** dropdown is set to **Select a function**. Further right, the **Function ARN** is listed as `arn:aws:lambda:us-east-1:928747700347:function:DVSA-ORDER-MANAGER`, and the **Description** is empty. The **Last modified** timestamp is **21 hours ago**, and the **Application** is **serverlessrepo-OWASP-DVSA**. Below the overview, the **Code source** section is visible, showing the **Code** tab selected. It includes buttons for **Open in Visual Studio Code** and **Upload from**. The bottom of the console shows the **EXPLORER** pane with the **DVSA-ORDER-MANAGER** file structure. The footer contains the copyright notice: "© 2026, Amazon Web Services, Inc. or its affiliates." and links for **Privacy**, **Terms**, and **Cookie preferences**.

FIGURE 13: Successful deployment of updated Lambda code



Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Vulnerable Dependencies	Lambda package should use maintained and safe dependencies only.	order-manager.js, package.json, npm ls, npm audit, AWS Console	Normal order-manager flow and successful Lambda deployment	npm audit showed critical and moderate issues in installed backend packages

Table B — Deviation analysis, fix, and verification

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Vulnerable Dependencies	The backend kept known vulnerable packages in a live Lambda package.	Misconfiguration	Removed node-serialize and node-jose from order-manager package; replaced decode call in order-manager.js	npm ls and npm audit were cleaner, and Lambda deployed successfully	Not measured

9.1 Intended Logic and Security Rule(s)

The intended logic is that the user sends a request to DVSA, API Gateway forwards it to the order-manager Lambda, and the Lambda processes the request using trusted code and dependencies. The correct outcome is normal order processing with safe backend libraries. The security rule is that the backend must not include dependencies with known critical vulnerabilities.

9.2 Evidence Sources and Behavior Trace

The main evidence came from source code search, package.json, npm ls, npm audit, the zip package, and AWS Console. In normal case, the Lambda package is expected to process requests correctly. In the vulnerable case, npm audit proved the Lambda package included unsafe dependencies. After the fix, the dependency list changed, the audit findings for these packages were removed, and the function was deployed again.

9.3 Deviation Analysis and Classification

This behavior is a deviation because the application should not keep vulnerable packages in a live backend function. The audit output is the proof of this deviation. I classify it as misconfiguration because the issue came from package management and not from intended business logic.

9.4 Explainable Fix and Post-Fix Validation

The wrong assumption was that old packages can stay in the project without risk if the code mostly works. The fix was applied in order-manager.js and its package files by removing node-serialize and node-jose, then using base64url for the needed decode operation. Post-fix validation was done with npm ls, npm audit, zip deployment, and a normal application test.

Part 10) Takeaway / Lessons Learned

The main problem was trusting old third-party packages in a serverless backend. This is important in Lambda because every deployment package becomes part of the attack surface. A good security principle here is to remove unused dependencies, keep packages updated, and use the smallest safe library needed for the task.